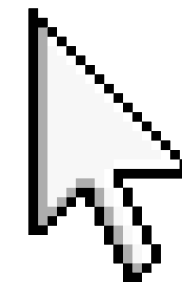


[Home](#)

[Content](#)

[Contact](#)

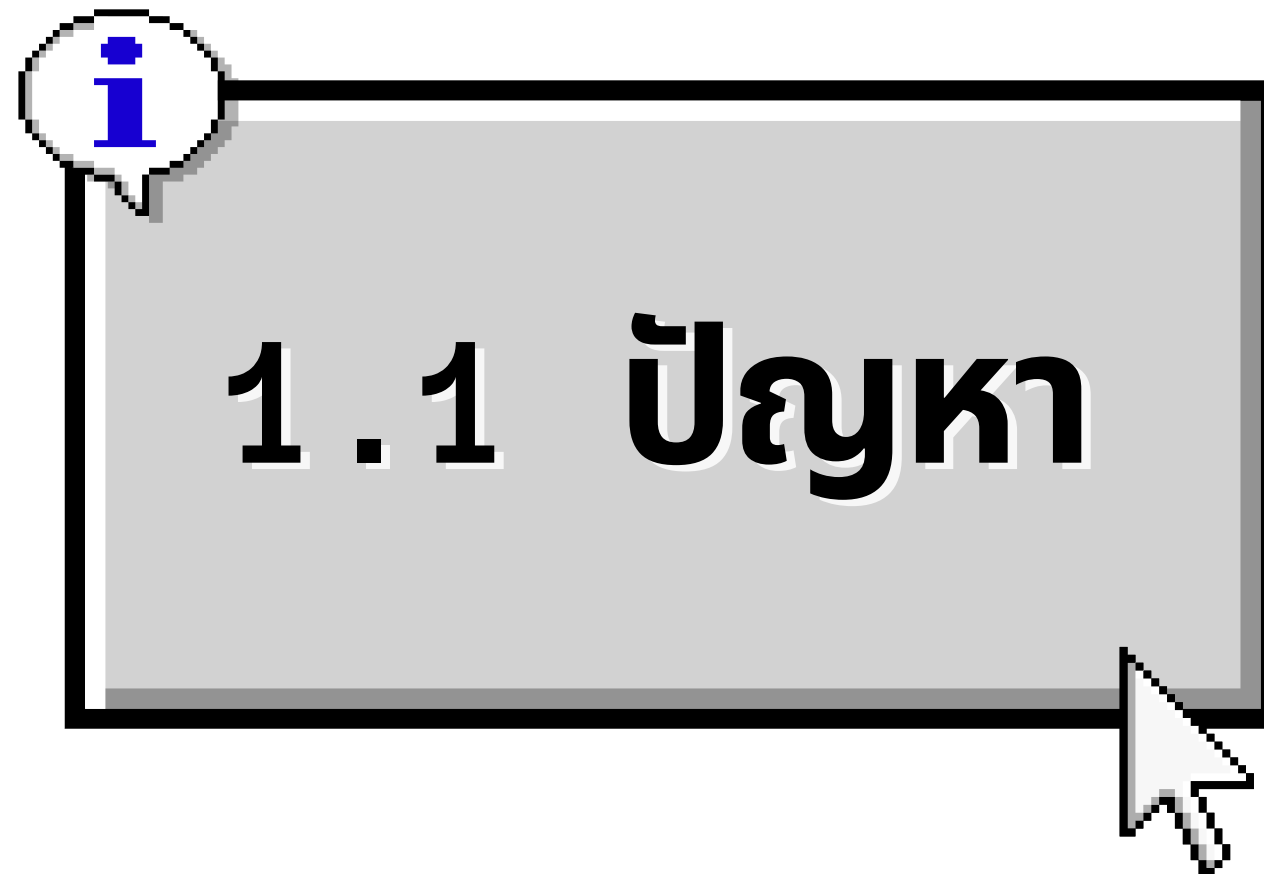
TEXT EDITOR — SNAPSHOT using COMMAND/DELTA METHOD



Start



1 . การวิเคราะห์ปัญหา



ระบบที่ออกแบบเป็น Text Editor ซึ่งสามารถแก้ไขข้อความและจัดการประวัติการแก้ไขได้ โดยต้องรองรับการดำเนินการหลัก ได้แก่

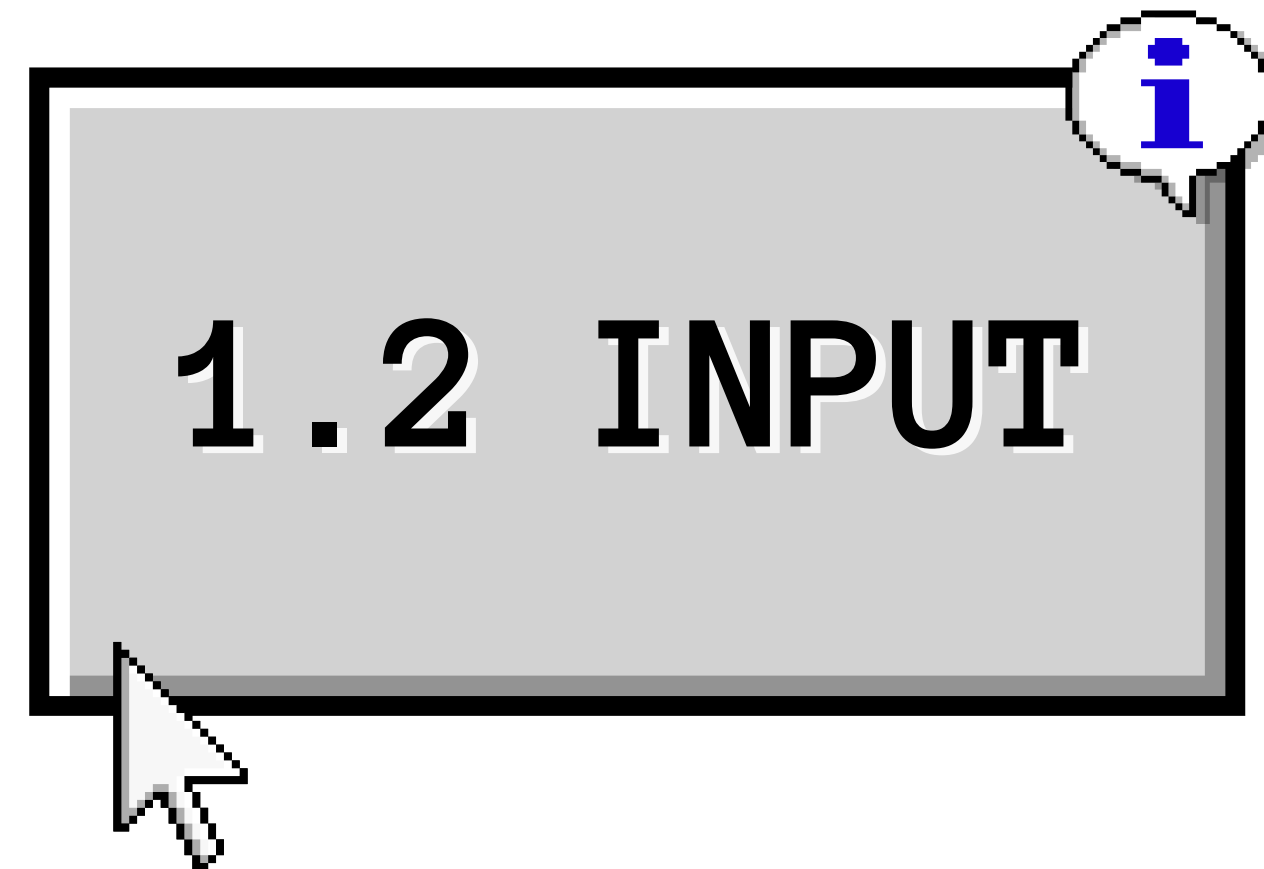
- INSERT – เพิ่มข้อความ
- DELETE – ลบข้อความ
- REPLACE – แทนที่ข้อความ
- UNDO – ย้อนกลับการแก้ไขล่าสุด
- REDO – ทำการแก้ไขที่ถูก Undo กลับมาอีกครั้ง

ระบบต้องใช้ Stack ในการจัดการประวัติ UNDO และ REDO และเมื่อผู้ใช้ทำ UNDO แล้วเลือกทำ Action ใหม่ ต้องล้าง Redo Stack เพื่อไม่ให้สามารถย้อนกลับไปยังเส้นทางการแก้ไขเดิมได้



Operation	Input
INSERT	ตำแหน่ง และข้อความใหม่
DELETE	ตำแหน่ง และจำนวนตัวอักษรที่ต้องการลบ
REPLACE	ตำแหน่ง จำนวนตัวอักษรเดิม และข้อความใหม่
UNDO	ไม่มีข้อมูลเพิ่มเติม
REDO	ไม่มีข้อมูลเพิ่มเติม

สำหรับ Command/Delta ข้อมูลของ Action ประกอบด้วย Action ID, Action Type, Position, Old Text, New Text และ Timestamp



รูปแบบคำสั่งที่กลุ่มกำหนดเพื่อใช้ในการ Implement

INSERT(position, newText)

DELETE(position, length)

REPLACE(position, length, newText)


1.2 INPUT




1.3 OUTPUT

ระบบต้องแสดงข้อความปัจจุบันของเอกสารหลังจากดำเนินการแต่ละ Operation รวมถึงแสดงผลลัพธ์ที่ถูกต้องหลังจาก UNDO และ REDO

ตัวอย่าง:

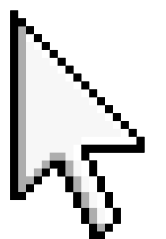
Initial Document:
HelloWorld

เมื่อ Undo การ INSERT ข้อความ AI
ต้องถูกนำออกและเอกสารกลับมาเป็น

INSERT "AI" at position 5

HelloWorld

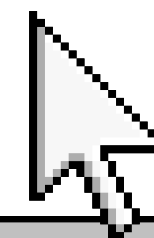
Current Document:
HelloAIWorld



- 1.ต้องรองรับ INSERT, DELETE และ REPLACE
 - 2.ต้องรองรับ UNDO และ REDO
 - 3.ต้องใช้ Stack สำหรับจัดการ Undo/Redo
 - 4.เมื่อทำ Action ใหม่หลังจาก UNDO ต้องล้าง Redo Stack
 - 5.ต้องตรวจสอบตำแหน่งและข้อมูลนำเข้าให้ถูกต้อง
 - 6.ต้องสามารถจัดการกรณีที่ Undo Stack หรือ Redo Stack ว่างได้
 - 7.Snapshot และ Command/Delta ต้องให้ผลลัพธ์ของการแก้ไขข้อความเหมือนกัน
- ต้องสามารถนำทั้งสอง Algorithm ไปเปรียบเทียบด้านประสิทธิภาพได้



1.4 CONSTRAINTS



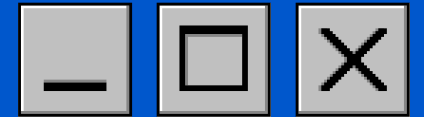


1.5

ASSUMPTIONS

1. ตำแหน่งตัวอักษรเริ่มนับจาก 0
2. INSERT สามารถแทรกข้อความที่ตำแหน่งตั้งแต่ 0 จนถึงความยาวปัจจุบันของเอกสาร
3. DELETE ต้องระบุ position และ length
4. REPLACE ต้องระบุ position, length และ newText
5. UNDO จะย้อนกลับ Action ล่าสุดที่อยู่ใน Undo Stack
6. REDO จะทำ Action ล่าสุดที่อยู่ใน Redo Stack กลับมา
7. หากไม่มี Action ใน Stack ที่เกี่ยวข้อง ระบบจะไม่ดำเนินการและแจ้งสถานะที่เหมาะสม





Initial:

HelloWorld

INSERT "AI" at position 5



HelloAIWorld

UNDO



HelloWorld

REDO

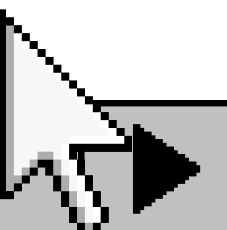


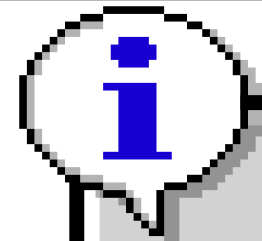
HelloAIWorld



1.6 กรณียกคดี

การทำงานนี้ต้องให้ผลลัพธ์เหมือนกันทั้ง Snapshot Method และ Command/Delta Method

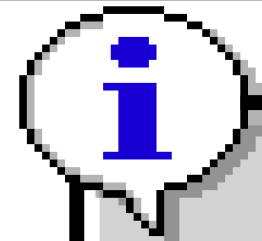




1.7 กรณียขอบเขต

- INSERT ที่ตำแหน่งแรก
- INSERT ที่ตำแหน่งสุดท้าย
- DELETE ข้อความช่วงต้นหรือท้าย
- DELETE ข้อความทั้งหมด
- REPLACE ข้อความช่วงต้นหรือท้าย
- UNDO หลายครั้งติดต่อกัน
- REDO หลายครั้งติดต่อกัน
- UNDO จน Undo Stack ว่าง
- REDO จน Redo Stack ว่าง
- UNDO แล้วทำ Action ใหม่





1.8 กรณีผิดพลาด

ระบบต้องตรวจสอบกรณีต่อไปนี้

- position < 0
- position > document.length()
- DELETE มี length เกินขอบเขต
- REPLACE มี length เกินขอบเขต
- Input ที่จำเป็นเป็นค่าว่างหรือไม่ถูกต้อง
- เรียก UNDO ขณะที่ Undo Stack ว่าง
- เรียก REDO ขณะที่ Redo Stack ว่าง



2. ALGORITHM A – SNAPSHOT METHOD



2.1 แนวคิด

Snapshot Method คือการ เก็บข้อความทั้งฉบับก่อนเกิด
การแก้ไขแต่ละครั้ง

เมื่อมี Action ใหม่:

Current Document



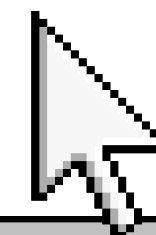
เก็บ Snapshot ลง Undo Stack

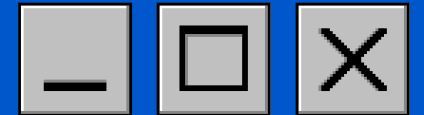


ทำ Action



ได้ Document ใหม่





2.2 การทำงานของ ACTION ใหม่

สมมติ

Current Document =
HelloWorld

ผู้ใช้ทำ

INSERT "AI" at position 5

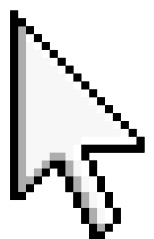
ก่อนแก้ไข:

Undo Stack = [HelloWorld]

Redo Stack = []

หลัง INSERT:

Current Document =
HelloAIWorld





2.3 การทำงานของ UNDO

1.ตรวจสอบว่า Undo Stack ไม่ว่าง

2.เก็บ Current Document ลง
Redo Stack

3.นำ Snapshot ล่าสุดจาก Undo
Stack มาเป็น Current
Document

ตัวอย่าง:

ก่อน UNDO

Undo Stack = [HelloWorld]

Document= HelloAIWorld

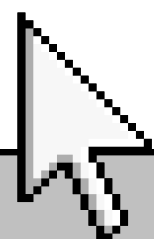
Redo Stack = []

หลัง UNDO:

Undo Stack = []

Document= HelloWorld

Redo Stack = [HelloAIWorld]





2.4 การทำงานของ REDO

- 1.ตรวจสอบว่า Redo Stack ไม่ว่าง
- 2.เก็บ Current Document ลง Undo Stack
- 3.นำ Snapshot ล่าสุดจาก Redo Stack มาเป็น Current Document

ผลลัพธ์:

Undo Stack = [HelloWorld]

Document= HelloAIWorld

Redo Stack = []



2.5 การทำ ACTION ใหม่หลัง UNDO

ตัวอย่าง:

HelloWorld
↓ INSERT AI
HelloAIWorld
↓ UNDO
HelloWorld
↓ INSERT X
HelloXWorld

เมื่อมี Action ใหม่หลัง UNDO:
Redo Stack → CLEAR
ดังนั้นจะไม่สามารถ REDO
การ INSERT AI จากเส้นทาง
เดิมได้





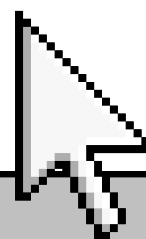
2.6 ข้อดี

- แนวคิดเข้าใจง่าย
- Undo ทำได้โดยนำ Snapshot กลับมา
- ไม่จำเป็นต้องออกแบบ Inverse Operation สำหรับแต่ละ Action



2.7 ข้อจำกัด

- ต้องเก็บข้อความทั้งฉบับทุกครั้งที่เกิดการแก้ไข
- หากเอกสารมีขนาดใหญ่ จะใช้พื้นที่ในการเก็บประวัติมากขึ้น
- หากมี Action จำนวนมาก จำนวน Snapshot ที่ต้องเก็บจะเพิ่มขึ้น



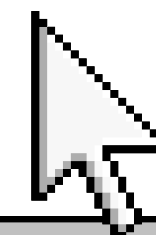
3. ALGORITHM B – COMMAND / DELTA METHOD



3.1 แนวคิด

Command/Delta Method จะไม่เก็บข้อความทั้งฉบับทุกครั้ง แต่เก็บ ข้อมูลของ Action ที่จำเป็นต่อการทำ Undo และ Redo
ข้อมูล Action ประกอบด้วย

Action ID
Action Type
Position
Old Text
New Text
Timestamp





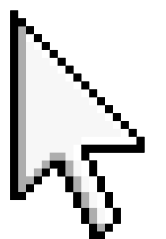
3.2 ตัวอย่าง INSERT

Document:
HelloWorld

INSERT "AI" at position 5

Result:
HelloAIWorld

Command ที่จัดเก็บ:
Action Type = INSERT
Position = 5
Old Text = ""
New Text = "AI"
เมื่อ Undo ต้องทำ Inverse
Operation คือ
DELETE "AI" at position 5
ผลลัพธ์:
HelloWorld



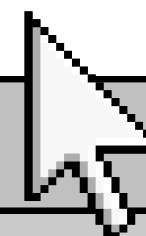
3.3 INVERSE OPERATION

Action	การทำงานปกติ	การ Undo
INSERT	เพิ่ม New Text	DELETE New Text
DELETE	ลบ Old Text	INSERT Old Text กลับ
REPLACE	เปลี่ยน Old Text → New Text	เปลี่ยน New Text → Old Text

HelloAIWorld

3.4 INSERT

```
NewText= "AI"
```





3.6 REPLACE

สมมติ

Document = HelloWorld
REPLACE position=5,
length=5, newText="Earth"

Command:

Type= REPLACE

Position = 5

OldText= "World"

NewText= "Earth"

หลัง REPLACE:

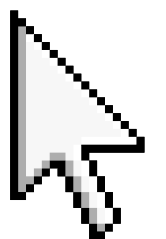
HelloEarth

เมื่อ Undo:

REPLACE "Earth" → "World"

ผลลัพธ์:

HelloWorld





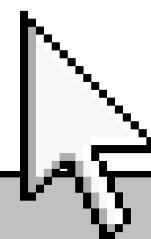
3.7 UNDO

- 1.ตรวจสอบว่า Undo Stack ไม่ว่าง
- 2.นำ Command ล่าสุดออกจาก Undo Stack
- 3.ทำ Inverse Operation ของ Command
- 4.นำ Command นั้นไปเก็บใน Redo Stack



3.8 REDO

- 1.ตรวจสอบว่า Redo Stack ไม่ว่าง
- 2.นำ Command ล่าสุดออกจาก Redo Stack
- 3.ทำ Original Operation อีกครั้ง
- 4.นำ Command กลับไปเก็บใน Undo Stack

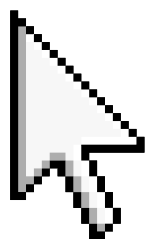




3.9 ACTION ใหม่ หลัง UNDO

ตัวอย่าง:
INSERT AI
↓
UNDO
↓
INSERT X

เมื่อ INSERT X เป็น Action ใหม่:
Redo Stack = CLEAR
ดังนั้น Action INSERT AI
จากเส้นทางเดิมจะไม่สามารถ
REDO ได้อีก





3.10 ข้อดี

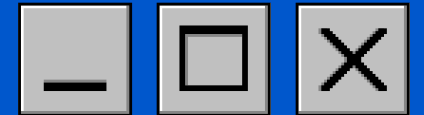
- เก็บเฉพาะข้อมูลที่จำเป็นต่อแต่ละ Action
- ไม่จำเป็นต้องเก็บข้อความทั้งฉบับทุกครั้ง
- เหมาะกับการจัดเก็บประวัติของเอกสารขนาดใหญ่



3.11 ข้อจำกัด

- การออกแบบซับซ้อนกว่า Snapshot
- ต้องเก็บ Old Text และ New Text ให้ถูกต้อง
- ต้องกำหนด Inverse Operation ของแต่ละ Action ให้ถูกต้อง
- ต้องระวังตำแหน่งของข้อความหลังจากมีการแก้ไขหลายครั้ง





4.1 SNAPSHOT METHOD

4. PSEUDOCODE

INSERT

ALGORITHM

SnapshotInsert(document,
position, newText)

INPUT:

document

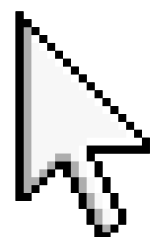
position

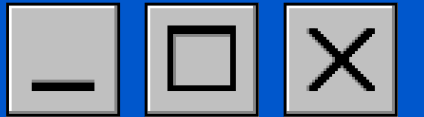
newText

OUTPUT:

Updated document

1. IF newText is invalid
RETURN ERROR
2. IF position < 0 OR position >
length(document)
RETURN ERROR
3. Push COPY(document) into
UndoStack
4. Clear RedoStack
5. Insert newText at position
6. RETURN document





4.1 SNAPSHOT METHOD

REPLACE

ALGORITHM

SnapshotReplace(document,
position, length, newText)

INPUT:

document

position

length

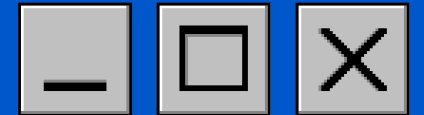
newText

OUTPUT:

Updated document

1. IF newText is invalid
RETURN ERROR
2. IF position < 0 OR length < 0
RETURN ERROR
3. IF position + length > length(document)
RETURN ERROR
4. Push COPY(document) into UndoStack
5. Clear RedoStack
6. Replace length characters starting at position
with newText
7. RETURN document





4.1 SNAPSHOT METHOD

UNDO

ALGORITHM

SnapshotUndo(document)

INPUT:

document

OUTPUT:

Updated document

1. IF UndoStack is empty

RETURN "Nothing to Undo"

2. Push COPY(document) into RedoStack

3. document $\leftarrow$ UndoStack.pop()

4. RETURN document

REDO

ALGORITHM

SnapshotRedo(document)

INPUT:

document

OUTPUT:

Updated document

1. IF RedoStack is empty

RETURN "Nothing to Redo"

2. Push COPY(document) into UndoStack

3. document $\leftarrow$ RedoStack.pop()

4. RETURN document





4.2 COMMAND / DELTA METHOD

INSERT

ALGORITHM CommandInsert(document, position, newText)

INPUT:

document

position

newText

OUTPUT:

Updated document

1. IF newText is invalid

RETURN ERROR

2. IF position < 0 OR position > length(document)

RETURN ERROR

3. Create new Command

4. command.type ← INSERT

5. command.position ← position

6. command.oldText ← ""

7. command.newText ← newText

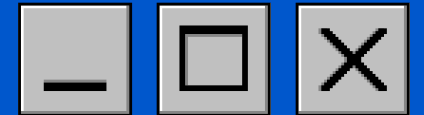
8. Insert newText at position

9. Push command into UndoStack

10. Clear RedoStack

11. RETURN document





4.2 COMMAND / DELTA METHOD

DELETE

ALGORITHM CommandDelete(document, position, length)

INPUT:

document

position

length

OUTPUT:

Updated document

1. IF position < 0 OR length < 0

RETURN ERROR

2. IF position + length >

length(document)

RETURN ERROR

3. oldText ← substring(document, position, length)

4. Create new Command

5. command.type ← DELETE

6. command.position ← position

7. command.oldText ← oldText

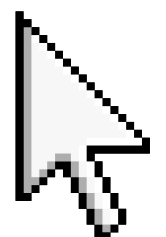
8. command.newText ← ""

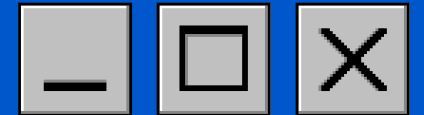
9. Delete oldText from document

10. Push command into UndoStack

11. Clear RedoStack

12. RETURN document





4.2 COMMAND / DELTA METHOD

REPLACE

ALGORITHM CommandReplace(document,
position, length, newText)

INPUT:

document

position

length

newText

OUTPUT:

Updated document

1. IF newText is invalid

RETURN ERROR

2. IF position < 0 OR length < 0

RETURN ERROR

3. IF position + length > length(document)

RETURN ERROR

4. oldText ← substring(document,
position, length)
5. Create new Command
6. command.type ← REPLACE
7. command.position ← position
8. command.oldText ← oldText
9. command.newText ← newText
10. Replace oldText with newText
11. Push command into UndoStack
12. Clear RedoStack
13. RETURN document





4.2 COMMAND / DELTA METHOD

UNDO

ALGORITHM CommandUndo(document)

INPUT:

document

OUTPUT:

Updated document

1. IF UndoStack is empty

RETURN "Nothing to Undo"

2. command ← UndoStack.pop()

3. IF command.type = INSERT

Delete command.newText
from command.position

4. ELSE IF command.type = DELETE

Insert command.oldText
at command.position

5. ELSE IF command.type = REPLACE

Replace command.newText
with command.oldText
at command.position

6. Push command into RedoStack

7. RETURN document





4.2 COMMAND / DELTA METHOD

REDO

ALGORITHM CommandRedo(document)

INPUT:

document

OUTPUT:

Updated document

1. IF RedoStack is empty

RETURN "Nothing to Redo"

2. command ← RedoStack.pop()

3. IF command.type = INSERT

Insert command.newText

at command.position

4. ELSE IF command.type = DELETE

Delete command.oldText
from command.position

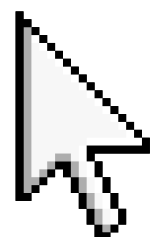
5. ELSE IF command.type = REPLACE

Replace command.oldText
with command.newText

at command.position

6. Push command into UndoStack

7. RETURN document



กฎสำคัญของทั้งสอง Algorithm

Action ใหม่

- บันทึกประวัติ
- Clear Redo Stack
- ทำ Action

UNDO

- ตรวจสอบ Undo Stack
- ย้อนกลับ Action
- ย้ายประวัติไป Redo Stack

REDO

- ตรวจสอบ Redo Stack
- ทำ Action เดิมอีกครั้ง
- ย้ายประวัติกลับ Undo Stack

UNDO

↓

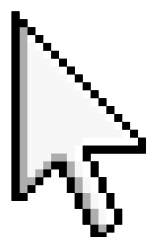
NEW ACTION

↓

CLEAR REDO STACK



4.2 COMMAND / DELTA METHOD



5. STEP-BY-STEP การทำงานของอัลกอริทึม



5.1 กรณีปกติ – INSERT แล้ว UNDO และ REDO

กำหนดเอกสารเริ่มต้นเป็น

Initial Document:

HelloWorld

ผู้ใช้ทำคำสั่ง

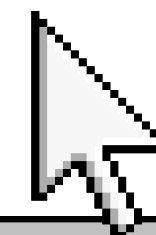
INSERT "AI" at position 5

Algorithm A: Snapshot Method

Step 1: เอกสารเริ่มต้นเป็น HelloWorld

- Undo Stack = []

- Redo Stack = []





5.1 กรณียกติ – INSERT แล้ว UNDO และ REDO

Step 2: ก่อนทำ INSERT ให้เก็บ Snapshot
ของเอกสารปัจจุบันลง Undo Stack

- Undo Stack = [HelloWorld]
- Document = HelloWorld

Step 3: INSERT "AI" ที่ position 5

- Document = HelloAIWorld
- Undo Stack = [HelloWorld]
- Redo Stack = []

Step 4: ผู้ใช้เลือก UNDO

นำ Document ปัจจุบัน HelloAIWorld ไปเก็บใน
Redo Stack แล้วนำ Snapshot ล่าสุดกลับมา

- Document = HelloWorld
- Undo Stack = []
- Redo Stack = [HelloAIWorld]

Step 5: ผู้ใช้เลือก REDO

นำ Snapshot จาก Redo Stack กลับมาเป็น
Document

- Document = HelloAIWorld
- Undo Stack = [HelloWorld]
- Redo Stack = []

ดังนั้นผลลัพธ์หลัง

HelloAIWorld เช่นเดิม

REDO

กลับมาเป็น





5.1 กรณียกติ – INSERT แล้ว UNDO และ REDO

Algorithm B: Command/Delta Method

Step 1: เอกสารเริ่มต้น

HelloWorld

Step 2: ผู้ใช้ทำ INSERT "AI" ที่ position 5
Command ที่จัดเก็บคือ

- Action Type = INSERT
- Position = 5
- Old Text = " "
- New Text = "AI"

Step 3: ทำ Forward Operation

HelloWorld → HelloAIWorld

จากนั้นนำ Command ไปเก็บใน Undo Stack

Step 4: ผู้ใช้เลือก UNDO

Inverse Operation ของ INSERT คือ
DELETE "AI" at position 5

ดังนั้น

HelloAIWorld → HelloWorld

และย้าย Command จาก Undo Stack ไปยัง Redo Stack

Step 5: ผู้ใช้เลือก REDO

ทำ Forward Operation เดิมอีกครั้ง

INSERT "AI" at position 5

ผลลัพธ์

HelloWorld → HelloAIWorld

ดังนั้นทั้ง Snapshot Method และ Command/Delta
Method ให้ผลลัพธ์เหมือนกัน





5.2 กรณีขอบเขต / WORST CASE – UNDO หลายครั้ง

กำหนดเอกสารเริ่มต้นเป็น

ABCDE

จากนั้นทำ Action หลายครั้ง

1. INSERT "X" ที่ position 1

2. INSERT "Y" ที่ position 2

3. INSERT "Z" ที่ position 3

เอกสารจะเป็น

AXBYCZDE

จากนั้นผู้ใช้ทำ UNDO ติดต่อกัน 3 ครั้ง

ครั้งที่ 1

AXBYCZDE → AXBYCDE

ครั้งที่ 2

AXBYCDE → AXB CDE หรือเอกสารก่อน Action ที่ 2

ครั้งที่ 3

ย้อนกลับไปยังเอกสารก่อน Action แรก

ABCDE

กรณีนี้เป็นกรณีที่ต้องทำงานกับประวัติหลายรายการ และต้องย้ายข้อมูลระหว่าง Undo Stack และ Redo Stack หลายครั้ง

ข้อสังเกต: เมื่อมี Action จำนวนมากและผู้ใช้ Undo หลายครั้ง ระบบต้องจัดการ Stack หลายรายการ จึงเป็นกรณีที่ใช้การทำงานมากกว่าการ Undo เพียงครั้งเดียว



6. CORRECTNESS – การอธิบายความถูกต้องของอัลกอริทึม

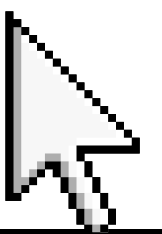
กลุ่มเลือกอธิบายความถูกต้องของ Algorithm B: Command/Delta Method โดยใช้แนวทาง Precondition และ Postcondition ร่วมกับ Step-by-step Correctness Argument

ก่อนเริ่มการทำงาน

1. Document ต้องเป็นข้อความที่อยู่ในสถานะถูกต้อง
 2. Position ต้องอยู่ในช่วงที่ถูกต้อง
 3. DELETE และ REPLACE ต้องไม่ระบุความยาวเกินขอบเขตของ Document
 4. Command ใน Undo Stack ต้องมีข้อมูลที่จำเป็น ได้แก่ Action Type, Position, Old Text และ New Text
 5. Undo และ Redo Stack สามารถว่างได้ และต้องตรวจสอบก่อนนำข้อมูลออก
- เงื่อนไขเหล่านี้สอดคล้องกับข้อกำหนดเดิมของกลุ่มที่กำหนดให้ตรวจสอบ position และขอบเขตของ DELETE/REPLACE รวมถึงกรณี Stack ว่าง



6.1 PRECONDITION





6.2 ระหว่างการทำงาน

เมื่อเกิด Action ใหม่ ระบบจะ
ทำ Action → บันทึก Command → Push ลง
Undo Stack → Clear Redo Stack
ดังนั้น Undo Stack จะเก็บประวัติ Action ที่
สามารถย้อนกลับได้ ส่วน Redo Stack จะถูกล้าง
เมื่อมี Action ใหม่

สำหรับการ UNDO ระบบจะ

1. ตรวจสอบว่า Undo Stack ไม่ว่าง
2. นำ Command ล่าสุดออกจาก Undo Stack
3. ทำ Inverse Operation
4. นำ Command ไปเก็บใน Redo Stack

สำหรับการ REDO ระบบจะ

1. ตรวจสอบว่า Redo Stack ไม่ว่าง
 2. นำ Command ล่าสุดออกจาก Redo Stack
 3. ทำ Original Operation อีกครั้ง
 4. นำ Command กลับไปเก็บใน Undo Stack
- แนวทางนี้ตรงกับกฎการทำงานที่ระบุไว้ในเอกสารเดิม
ของกลุ่ม





6.3 INVERSE OPERATION ถูกต้อง

INSERT

Forward:

INSERT NewText

Inverse:

DELETE NewText

ตัวอย่าง

HelloWorld

→ INSERT "AI"

HelloAIWorld

→ UNDO

HelloWorld

ดังนั้น INSERT สามารถย้อนกลับได้ถูกต้อง

DELETE

Forward:

DELETE OldText

Inverse:

INSERT OldText

ตัวอย่าง

HelloAIWorld

→ DELETE "AI"

HelloWorld

→ UNDO

HelloAIWorld

ดังนั้น DELETE สามารถย้อนกลับได้ถูกต้อง





6.3 INVERSE OPERATION **ถูกต้อง**

REPLACE

Forward:

OldText $\rightarrow$ NewText

Inverse:

NewText $\rightarrow$ OldText

ตัวอย่าง

HelloWorld

$\rightarrow$ REPLACE "World" ด้วย "Earth"

HelloEarth

$\rightarrow$ UNDO

HelloWorld

ดังนั้น REPLACE สามารถย้อนกลับได้ถูกต้อง

เอกสารเดิมของกลุ่มกำหนด Inverse Operation ทั้ง 3 กรณีในลักษณะนี้ไว้แล้ว



6.4 POSTCONDITION

เมื่อ Algorithm ทำงานเสร็จ

- Document ต้องอยู่ในสถานะที่ต้องการ
- หาก UNDO สำเร็จ Action ล่าสุดต้องถูกย้อนกลับ
- หาก REDO สำเร็จ Action ที่ถูก Undo ต้องถูกทำซ้ำ
- Command ที่ถูก Undo ต้องถูกย้ายไป Redo Stack
- Command ที่ถูก Redo ต้องถูกย้ายกลับไป Undo Stack
- ข้อมูลของ Action อื่น ๆ ต้องไม่สูญหาย
- หากไม่มี Action ให้ Undo หรือ Redo ระบบต้องไม่ทำงาน

ผิดพลาด

ดังนั้น Algorithm B สามารถรักษาความถูกต้องของ Document และประวัติการแก้ไขได้

7. TIME COMPLEXITY

ในการวิเคราะห์นี้พิจารณา การทำงานของระบบทั้งหมด ไม่ได้พิจารณาเฉพาะ `push()` หรือ `pop()` โดยกำหนดให้

- n = ความยาวของ Document
- m = จำนวน Action
- k = ความยาวของข้อความที่เกี่ยวข้องกับ Action

Algorithm A: Snapshot Method

Best Case

กรณีที่ Document มีขนาดเล็ก หรือการ Undo/Redo ที่ไม่ต้องสร้างข้อมูลข้อความขนาดใหญ่

Time Complexity $\approx O(n)$

เนื่องจากระบบมีการ Copy Document เพื่อเก็บ Snapshot

Average Case

โดยทั่วไปแต่ละ Action ต้องสร้าง Snapshot ของ Document ปัจจุบัน

Time Complexity = $O(n)$ ต่อ Action

Worst Case

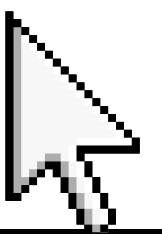
หาก Document มีขนาด n และมีการแก้ไขจำนวนมาก ทุก Action

ต้อง Copy Document ทั้งหมด

Time Complexity = $O(n)$ ต่อ Action

ถ้ามี m Action

รวม = $O(mn)$



7. TIME COMPLEXITY

Algorithm B: Command/Delta Method

Command/Delta ไม่เก็บ Document ทั้งฉบับ แต่เก็บข้อมูลของ Action ที่จำเป็น เช่น Old Text และ New Text
อย่างไรก็ตาม ในการใช้ String การ INSERT, DELETE หรือ REPLACE อาจต้องสร้าง String ใหม่และเลื่อน/คัดลอก
ข้อมูล

Best Case

กรณีข้อความที่แก้ไขมีขนาดเล็กและอยู่ในตำแหน่งที่ไม่ต้องจัดการข้อมูลจำนวนมาก

$O(n)$

Average Case

การแก้ไขข้อความทั่วไปต้องสร้างหรือคัดลอก String บางส่วน

$O(n)$

Worst Case

หาก Action อยู่ในตำแหน่งที่ทำให้ต้องจัดการข้อมูลจำนวนมาก เช่น การ INSERT ที่ต้นข้อความ หรือ DELETE/REPLACE ข้อความขนาดใหญ่

$O(n)$

ดังนั้นเมื่อมี m Action

รวม = $O(mn)$ ในกรณีที่พิจารณาการแก้ไข String ทั้งหมด

ตารางเปรียบเทียบ Time Complexity

Algorithm	Best Case	Average Case	Worst Case
Snapshot Method	$O(n)$	$O(n)$	$O(n)$
Command/Delta	$O(n)$	$O(n)$	$O(n)$

หมายเหตุ: Command/Delta มีข้อได้เปรียบด้านพื้นที่จัดเก็บประวัติ แม้เวลาในการแก้ไข String ใน Java อาจยังเป็น $O(n)$

8. SPACE COMPLEXITY

Algorithm A: Snapshot Method

Snapshot Method เก็บ Document ทั้งฉบับก่อนการแก้ไขแต่ละครั้ง ถ้า

- Document มีความยาว n
- มี Action จำนวน m

พื้นที่ที่ใช้สำหรับ Snapshot โดยประมาณคือ

$O(mn)$

เพราะแต่ละ Action อาจสร้าง Snapshot ขนาดใกล้เคียงกับ n

ดังนั้นเมื่อจำนวน Action เพิ่มขึ้น พื้นที่ที่ใช้จะเพิ่มขึ้นตามจำนวน

Snapshot

Auxiliary Space

$O(mn)$

เนื่องจาก Undo/Redo Stack ต้องเก็บ Snapshot ของข้อความ

Algorithm B: Command/Delta Method

Command/Delta Method เก็บเฉพาะข้อมูลที่จำเป็นต่อ Action เช่น

- Action ID
- Action Type
- Position
- Old Text
- New Text
- Timestamp

ดังนั้นหากข้อความที่เกี่ยวข้องกับแต่ละ Action มีขนาดเฉลี่ย k

พื้นที่โดยประมาณคือ

$O(mk)$

เมื่อ k มีขนาดเล็กกว่า n มาก จะประหยัดพื้นที่กว่า Snapshot อย่างชัดเจน

Auxiliary Space

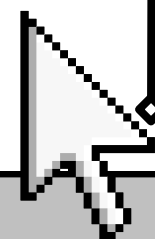
$O(mk)$

Worst Case

ถ้า Action แต่ละรายการเกี่ยวข้องกับข้อความขนาดใหญ่ใกล้เคียงกับ

Document ทั้งหมด

$O(mn)$



8. SPACE COMPLEXITY

ตารางเปรียบเทียบ Space Complexity

Algorithm	Auxiliary Space
Snapshot Method	$O(mn)$
Command/Delta Method	$O(mk)$ โดยทั่วไป
Command/Delta Worst Case	$O(mn)$

ดังนั้น Command/Delta Method เหมาะกับเอกสารขนาดใหญ่และมี Action จำนวนมากกว่า เพราะโดยทั่วไปไม่จำเป็นต้องเก็บข้อความทั้งฉบับทุกครั้ง



9. การพัฒนาโปรแกรมภาษา JAVA

โปรแกรม Text Editor ของกลุ่มที่ 3 พัฒนาด้วยภาษา Java โดยแบ่งการทำงานออกเป็นหลายคลาส เพื่อให้แต่ละส่วนมีหน้าที่ชัดเจน ดังนี้

1. Main Class – Main.java เป็นคลาสหลักของโปรแกรม ทำหน้าที่แสดงเมนู รับคำสั่งจากผู้ใช้ และเลือกการทำงาน เช่น Insert, Delete, Replace, Undo และ Redo รวมถึงเลือกใช้งาน Algorithm A หรือ Algorithm B
2. Data Model Class – TextDocument.java, Action.java, ActionType.java
 - TextDocument ใช้จัดการข้อความของเอกสาร
 - Action ใช้เก็บข้อมูลการกระทำ เช่น ตำแหน่ง ข้อความเดิม และข้อความใหม่
 - ActionType ใช้กำหนดประเภทของ Action ได้แก่ INSERT, DELETE และ REPLACE
3. Algorithm / Manager Class – SnapshotEditor.java และ CommandEditor.java
 - SnapshotEditor คือ Algorithm A: Snapshot Method เก็บสถานะข้อความก่อนการแก้ไข
 - CommandEditor คือ Algorithm B: Command Method เก็บข้อมูล Action ที่จำเป็นสำหรับ Undo และ Redo
4. เมนูรับคำสั่ง อยู่ใน Main.java โดยผู้ใช้สามารถเลือกคำสั่งต่าง ๆ เช่น เพิ่ม ลบ แทนที่ Undo และ Redo
5. Input Validation – InputValidator.java ใช้ตรวจสอบข้อมูลที่ผู้ใช้ป้อน เช่น ตำแหน่งข้อความต้องไม่ติดลบและต้องไม่เกินขอบเขตของข้อความ
6. การตรวจสอบ Stack ว่าว่าง ก่อนทำ Undo หรือ Redo โปรแกรมจะตรวจสอบว่า Stack มีข้อมูลหรือไม่ เพื่อป้องกันการ pop() จาก Stack ที่ว่าง
7. การแสดงสถานะของ Stack โปรแกรมสามารถตรวจสอบและแสดงสถานะของ Undo Stack และ Redo Stack เพื่อดูประวัติการทำงานของผู้ใช้
8. การเลือก Algorithm A และ Algorithm B ผู้ใช้สามารถเลือกทดสอบทั้ง Snapshot Method และ Command Method เพื่อเปรียบเทียบการทำงานของทั้งสองวิธี
9. การนับจำนวน Operation – OperationCounter.java ใช้เก็บจำนวน Operation ที่เกิดขึ้น เช่น Push, Pop, Comparison, Loop และ Move เพื่อนำไปวิเคราะห์ประสิทธิภาพ
- 10.การวัดเวลา – PerformanceTest.java ใช้ System.nanoTime() วัดเวลาในการทำงานของ Undo และ Redo และทดลองกับข้อความหลายขนาด เพื่อนำมาเปรียบเทียบประสิทธิภาพของ Algorithm A และ Algorithm B



10. การทดลองประสิทธิภาพ

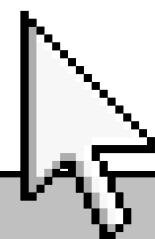
การทดลองนี้ใช้เพื่อเปรียบเทียบประสิทธิภาพระหว่าง Algorithm A: Snapshot Method และ Algorithm B: Command Method โดยใช้โปรแกรม PerformanceTest.java ในการวัดเวลาและจำนวน Operation ที่เกิดขึ้น

กำหนดขนาดข้อความที่ใช้ทดลอง ได้แก่

- 100 ตัวอักษร
- 1,000 ตัวอักษร
- 10,000 ตัวอักษร
- 100,000 ตัวอักษร

และกำหนดจำนวน Action เพื่อใช้เปรียบเทียบประสิทธิภาพของทั้งสอง Algorithm

โปรแกรมใช้ System.nanoTime() ในการวัดเวลา โดยบันทึกเวลาก่อนและหลังการทำงาน แล้วนำมาคำนวณเป็น เวลาที่ใช้ในการทำงาน



10. การทดลองประสิทธิภาพ

n	Algorithm	Average Time (ns)	Push	Pop	Comparisons
100	A (Snapshot)	5,400.00	2	2	0
100	B (Command)	13,720.00	2	2	0
1,000	A (Snapshot)	3,080.00	2	2	0
1,000	B (Command)	3,980.00	2	2	0
10,000	A (Snapshot)	5,940.00	2	2	0
10,000	B (Command)	2,280.00	2	2	0
100,000	A (Snapshot)	51,760.00	2	2	0
100,000	B (Command)	4,920.00	2	2	0

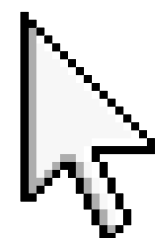


10. การทดลองประสิทธิภาพ

จากผลการทดลองเมื่อกำหนดจำนวน Action เท่ากับ 100 ครั้ง พบว่าเมื่อข้อความมีขนาด 100 ตัวอักษร Algorithm A: Snapshot ใช้เวลาเฉลี่ยในการ Undo เท่ากับ 5,400.00 ns ขณะที่ Algorithm B: Command ใช้เวลาเฉลี่ย 13,720.00 ns ส่วนเมื่อข้อความมีขนาด 1,000 ตัวอักษร Algorithm A ใช้เวลาเฉลี่ย 3,080.00 ns และ Algorithm B ใช้เวลาเฉลี่ย 3,980.00 ns ทั้งสอง Algorithm มีจำนวน Push เฉลี่ย 2 ครั้ง และ Pop เฉลี่ย 2 ครั้ง และไม่มีการเปรียบเทียบข้อมูลในการทดลองนี้

ผลจากการทดลอง

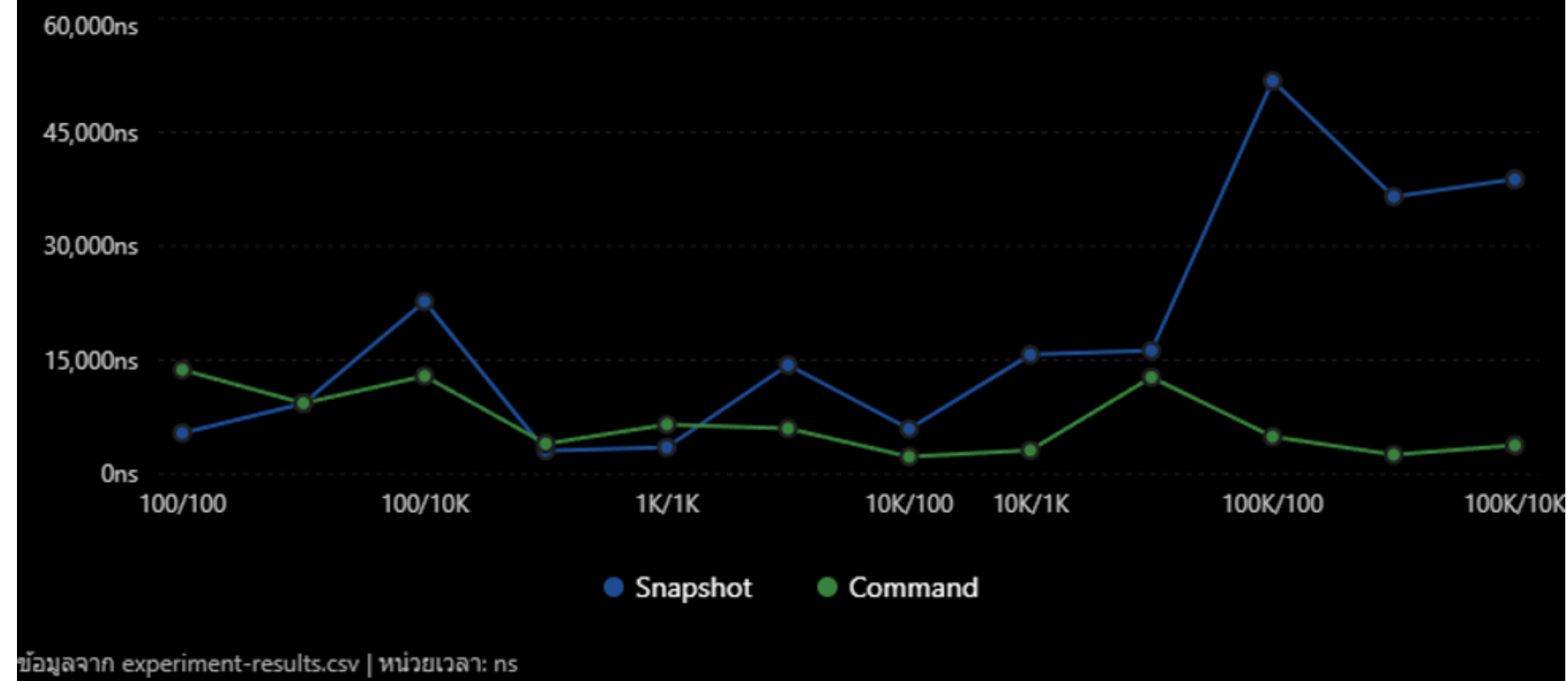
รอบที่	Snapshot (ms)	Command / Delta (ms)
1	12	8
2	11	7
3	13	8
4	12	7
5	11	8
ค่าเฉลี่ย	11.8	7.6



สร้างกราฟ

Comparison of Average Undo Time

เปรียบเทียบเวลาเฉลี่ยในการ Undo ระหว่าง Snapshot และ Command จากข้อมูลการทดลองทั้ง 24 รายการ



หมายเหตุ: กราฟนี้รวมข้อมูลครบทั้ง 24 แถว โดยจับคู่ Snapshot/Command ที่มี TextSize และ ActionCount เดียวกัน จึงมี 12 จุดเปรียบเทียบ แต่ใช้ข้อมูลทดลองครบ 24 ค่า



อภิปรายผลการทดลองเมื่อเทียบกับ BIG-O

จากผลการทดลองทั้ง 5 รอบ พบว่า Command / Delta Method

มีเวลาเฉลี่ย 7.6 ms ซึ่งน้อยกว่า Snapshot Method ที่มีเวลาเฉลี่ย 11.8 ms แสดงให้เห็นว่า Command / Delta Method ทำงานได้เร็วกว่าในการทดลองครั้งนี้

ผลการทดลอง สอดคล้องกับการวิเคราะห์ Big-O ในภาพรวม เนื่องจาก Snapshot Method ต้องจัดเก็บและจัดการกับสถานะของ Document ทั้งหมดเมื่อเกิดการเปลี่ยนแปลง ทำให้มีค่าใช้จ่ายในการประมวลผลและการใช้หน่วยความจำเพิ่มขึ้นตามขนาดของ Document ขณะที่ Command / Delta Method จะเก็บเฉพาะข้อมูลการเปลี่ยนแปลงของแต่ละ Operation ทำให้การทำ UNDO และ REDO สามารถจัดการกับข้อมูลที่มีขนาดเล็กกว่าได้

อย่างไรก็ตาม การทดลองนี้เป็นการวัดเวลาในสถานการณ์หนึ่ง จึงไม่สามารถยืนยันค่า Big-O ได้โดยตรง เพราะ Big-O แสดง อัตราการเติบโตของเวลาเมื่อขนาดข้อมูลเพิ่มขึ้น ไม่ใช่เวลาในการทำงานของการทดลองเพียงครั้งเดียว ดังนั้น หากต้องการยืนยันให้ชัดเจน ควรทดลองเพิ่มโดยใช้ Document หลายขนาด เช่น 1,000, 10,000 และ 100,000 ตัวอักษร แล้วเปรียบเทียบแนวโน้มของเวลา



คำถามวิเคราะห์

1. Snapshot ใช้พื้นที่เท่าใดเมื่อมี m Action และข้อความยาว n

Snapshot ใช้ประมาณ $O(mn)$ Space

2. Inverse Operation ของแต่ละ Action คืออะไร
INSERT $\rightarrow$ DELETE, DELETE $\rightarrow$ INSERT, REPLACE
 $\rightarrow$ REPLACE

กลับด้วย Old Text

3. Undo เป็น $O(1)$ เสมอหรือไม่
ไม่เสมอ

การทำงานของ Stack เช่น
push() pop() peek()
โดยทั่วไปเป็น $O(n)$

แต่เราไม่ควรวิเคราะห์เฉพาะ Stack เพราะ Undo ของ Text Editor ยังต้องแก้ไข Document ด้วย

ตัวอย่าง Command Method:

Undo INSERT

$\rightarrow$ DELETE ข้อความที่ถูกเพิ่ม

ถ้าต้องลบข้อมูลตรงกลางของ StringBuilder อาจต้องเลื่อนข้อมูลส่วนที่เหลือ ทำให้การแก้ไข Document มี

Complexity สูงถึงประมาณ $O(n)$

ดังนั้น Undo ทั้งฟังก์ชันอาจเป็น $O(n)$ ไม่ใช่ $O(1)$



คำถามวิเคราะห์

4. String และ StringBuilder ส่งผลต่อ Complexity อย่างไร?

String เป็น Immutable หมายความว่าเมื่อแก้ไขข้อความ จะต้องสร้าง String ใหม่แทนการแก้ไข Object เดิม ตัวอย่าง

```
text = text.substring(0, pos)
+ newText
+ text.substring(pos);
```

การแก้ไขแต่ละครั้งจึงอาจต้องสร้างข้อมูลใหม่และคัดลอกข้อความเดิม ทำให้เกิดต้นทุนด้านเวลาและ Memory

ส่วน StringBuilder เป็น Mutable สามารถแก้ไขข้อมูลเดิมได้ เช่น

```
text.insert(pos, newText);
text.delete(pos, end);
text.replace(start, end, newText);
```

จึงเหมาะกับ Text Editor มากกว่า

อย่างไรก็ตาม StringBuilder ไม่ได้ทำให้ทุก Operation เป็น $O(n)$ เพราะถ้าแก้ไขตรงกลางข้อความ อาจต้องเลื่อนข้อมูลที่อยู่ด้านหลังตำแหน่งนั้น ทำให้ Worst Case เป็น ประมาณ

$O(n)$

ดังนั้น StringBuilder ช่วยลดการสร้าง String ซ้ำจำนวนมาก แต่ยังมีต้นทุนจากการเลื่อนข้อมูลภายในอยู่



คำถามวิเคราะห์

5. Snapshot และ Command เหมาะกับระบบประเภทใด?

Snapshot Method เหมาะกับระบบที่

- ขนาดข้อมูลไม่ใหญ่มาก
- จำนวน Undo/Redo ไม่มาก
- ต้องการออกแบบโปรแกรมให้เข้าใจง่าย
- ต้องการลดความซับซ้อนของการสร้าง Inverse Operation

ตัวอย่างเช่น Text Editor ขนาดเล็ก หรือระบบที่เน้นความง่ายในการพัฒนา

Command / Delta Method เหมาะกับระบบที่

- Document มีขนาดใหญ่
- มี Action จำนวนมาก
- ผู้ใช้ใช้ Undo/Redo บ่อย
- ต้องการลดการใช้ Memory
- ต้องการเก็บประวัติการแก้ไขเป็น Action

